Proteus: A Morpheus II Extension for System-Level Moving-Target Defense and Key-Exposure Hardening

Jeremy Knickerbocker*, Syed Rafay Hasan†, Amr Hilal*, and Mohammad Ashiqur Rahman‡,
*Department of Computer Science, Tennessee Tech University, Cookeville, TN 38505 USA
†Department of Electrical and Computer Engineering, Tennessee Tech University, Cookeville, TN 38505 USA
‡Department of Electrical and Computer Engineering, Florida International University, Miami, FL 33199 USA

2026 27th International Symposium on Quality Electronic Design (ISQED) | 979-8-3315-8361-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/ISQED69900.2026.11534731

***Abstract*—Lightweight block ciphers, such as SIMON, are widely used in embedded and Internet of Things (IoT) systems that must balance security with limited hardware and energy budgets. The Morpheus II processor employs a 12-round SIMON core to protect architectural metadata and support moving-target defense (MTD). However, the core encrypts only at startup or upon fault detection, leaving long intervals where ciphertext in memory remains static. In Supervisory Control and Data Acquisition (SCADA)-connected IoT devices, adversaries with read and write access to main memory can exploit these intervals to analyze or manipulate encrypted state, leaving Morpheus II vulnerable to prolonged observation and control-flow manipulation. This work modifies the Morpheus II encryption core to reintroduce proactive churn, making Morpheus II suitable for IoT environments. The modified core, called Proteus, retains SIMON's Feistel structure and key schedule but replaces its fixed round function with predefined rotation sets selected by a system-controlled per-epoch schedule. Rotation sets can be mapped to memory regions, enabling cipher-level MTD. Initial cryptanalysis evaluation shows configuration-dependent improvements in differential behavior relative to the 12-round SIMON baseline, at a modest cost of area and performance.**

***Index Terms*—SIMON, Proteus, lightweight cipher, moving-target defense, FPGA, Internet of Things**

I. Introduction

The rapid expansion of embedded and Internet of Things (IoT) systems has created vast networks of resource-constrained devices that operate with limited computational, memory, and energy resources. These devices often manage sensitive operational data, control physical processes, and interact with critical infrastructure, making data confidentiality a primary security concern. Conventional ciphers such as AES [1], while robust, are often impractical for small microcontrollers and custom hardware because of their requirement for prohibitively enormous compute, latency, and, consequently, power consumption. To address these constraints, the cryptographic community has developed a range of lightweight block ciphers that achieve acceptable security margins with minimal hardware cost.

Among these designs, SIMON and SPECK [2] are commonly studied lightweight symmetric encryption algorithms which were introduced by the U.S. National Security Agency (NSA). For hardware designs, SIMON (SPECK is the software equivalent) is one of the most studied lightweight block ciphers [3]. SIMON's Feistel structure and use of simple bitwise operations such as rotation, AND, and XOR allow compact datapaths that can be efficiently mapped onto field-programmable gate arrays (FPGAs) or application-specific integrated circuits (ASICs). These properties make SIMON a common baseline for lightweight encryption. However, reduced-round variants used to meet hardware constraints provide narrower security margins against differential and linear cryptanalysis [4]–[14], particularly when the cipher structure remains fixed across all operations.

In parallel with the development of lightweight cryptography, the concept of *moving-target defense* (MTD) has gained traction as a system-level countermeasure to reverse engineering and control-flow manipulation [15]. MTD techniques periodically change key parameters, encodings, or instruction semantics to deny attackers a stable view of the system. When integrated at the hardware layer, MTD can frustrate attacks that depend on static architectural state or predictable memory content. In [16] and [17], the Morpheus research prototypes demonstrate this approach concretely in hardware. Morpheus I [16] employed fine-grained randomization of instruction encodings, pointers, and architectural metadata to achieve proactive churn at the microarchitectural level. By continuously changing internal representations, it prevented attackers from constructing reliable exploit payloads. Morpheus II [17] built on this foundation by introducing a standardized cryptographic primitive to secure architectural metadata, replacing the ad hoc randomization mechanisms used previously. Specifically, Morpheus II adopted a 12-round SIMON core for metadata encryption, chosen for its compact hardware footprint and reproducibility. Morpheus II was originally demonstrated on a RISC-V platform, but, to the best of the authors' understanding, the approach is applicable to other embedded environments. A drawback of the Morpheus II core is that during normal operation, the encryption core only churns at startup or when a fault is detected. Between these events, ciphertext in memory remains static under a fixed key and round structure.

For many general-purpose computing platforms, this reactive churn model offers adequate protection against short-lived or transient attacks. However, IoT and Supervisory Control and Data Acquisition (SCADA)-connected embedded systems,

Authorized licensed use limited to: University of Ulsan. Downloaded on July 17,2026 at 02:19:57 UTC from IEEE Xplore. Restrictions apply.

such as those deployed in Smart Grid infrastructures, operate continuously for extended periods, often unattended and exposed to untrusted network environments [18]. In these settings, an adversary may obtain persistent read and write access to main memory through remote exploitation or compromised firmware interfaces. When ciphertext remains static for long intervals, the attacker can observe and correlate encrypted and unencrypted data over time, revealing structural patterns or inferring plaintext. In addition, control-flow manipulation attacks can relocate encrypted code or data between memory regions without triggering re-encryption, allowing unauthorized execution or bypassing integrity mechanisms. These conditions motivate the need for proactive churn that periodically and autonomously re-encrypts system state, ensuring that the protected data and instruction streams remain a moving target throughout normal operation.

Research Questions. This work investigates how structural variability can strengthen the encryption mechanism used in the Morpheus II security extension while preserving its lightweight design constraints. The research is guided by the following questions:

1) To what extent can introducing per-epoch variability within a lightweight encryption core mitigate ciphertext reuse and instruction-reuse attacks that lead to control-flow manipulation in Morpheus II?
2) How can separating key and selector information, together with region-aware encryption, improve resistance to key exposure without violating hardware or architectural efficiency constraints?
3) Can these modifications be implemented within the hardware efficiency envelope of SIMON while meeting or exceeding its security margins at equivalent round counts?

Contributions of this paper: The design goal of *Proteus*[1] is to strengthen security against passive observation and key-exposure attacks without altering interfaces or significantly increasing hardware cost. In support of this goal, the paper makes the following contributions:

1) Introduces Proteus, a modified 12-round SIMON implementation with per-epoch rotation-set selection under system control.
2) Enables proactive churn and memory-region-aware encryption, increasing resistance to key exposure and control-flow manipulation.
3) Quantifies the resulting hardware and performance trade-offs using an Intel MAX10 FPGA implementation.

The modified core, called Proteus, preserves SIMON's Feistel structure and key schedule [2] but replaces its fixed round function with multiple predefined rotation sets. Differential evaluation under the same windowed analysis model shows that the 12-round SIMON (1,8,2) baseline exhibits mean strength 21.37 (minimum 20.22, maximum 26.10), while Proteus-2 using rotation sets (1,4,5) and (1,5,4) produces configuration-dependent results. Across the 16 evaluated rotation-set configuration keys, the Proteus mean strength ranges from 17.17 to 25.32, the minimum from 16.72 to 24.32, and the maximum from 18.43 to 30.32. Several configurations exceed the SIMON baseline, particularly in the upper bound, while other set of configuration fall below it, reflecting schedule-dependent symmetry effects. The rotation key selection that are weaker than the baseline Simon can be excluded, resulting in better security for our proposed Proteus design. Selector-controlled rotation sets expand the per-epoch schedule space from a single fixed structure to $2^{12} = 4{,}096$ schedules for Proteus-2 and $4^{12} = 16{,}777{,}216$ for Proteus-4. FPGA results on an Intel MAX10 device show that these structural gains come at a modest cost, with about 32% more lookup tables (LUTs) and roughly 13% lower throughput relative to the Morpheus II SIMON baseline.

The remainder of this paper is organized as follows. Section II describes the threat model and security goals. Section III outlines the system design and underlying framework. Section IV develops a system-level security model based on churn cost and rotation diversity. Section V presents the evaluation results. Section VI provides discussion and analysis, and Section VII concludes the paper.

II. Threat Model and Security Goals

A. Threat Model

System context. The target platform follows the Morpheus II architecture, integrating the **Proteus Encryption and Churn Unit (PECU)** within a RISC-V processor core shown in Fig. 1. Execution proceeds in epochs, during which all off-chip memory contents remain encrypted. Each epoch employs a unique encryption key and selector schedule derived from on-chip entropy. The processor, caches, and memory controller handle ciphertext exclusively outside the trusted hardware boundary.

Attacker information. The attacker is a remote operator connected through the SCADA or supervisory application interface, as illustrated in Fig. 1. Through exploitation of software vulnerabilities, the attacker gains persistent *read and write* permissions to main memory and peripheral interfaces. The attacker can therefore observe ciphertext stored in memory or transiting the system bus but never has direct access to plaintext. The attacker is assumed to have static knowledge of the *Proteus* cipher design, including its rotation sets, and can submit chosen plaintexts through standard SCADA interfaces.

Selector schedule exposure. The per-epoch selector schedule, which determines the round-level rotation structure, is generated on-chip and retained within the PECU trusted boundary. It is not written to off-chip memory and is not exposed through external interfaces. The attacker is assumed to know the overall cipher design and the predefined rotation sets, but not the active selector schedule used in a given epoch. For cryptanalytic evaluation, we analyze multiple selector configurations independently, treating each configuration as a

[1] *Proteus* is named after the shape-shifting sea god of Greek mythology, symbolizing adaptability and continual transformation, reflecting the cipher's dynamic moving-target design.

Authorized licensed use limited to: University of Ulsan. Downloaded on July 17,2026 at 02:19:57 UTC from IEEE Xplore. Restrictions apply.

deterministic instance of the cipher in order to characterize differential behavior across the schedule space.

Target of attack. The attacker's objective is to subvert normal device behavior by manipulating encrypted runtime state or control metadata. Specific aims that are addressed here include:

- Reusing, relocating, or modifying ciphertext blocks to trigger unauthorized execution paths;
- Conducting long-term observation of ciphertext to infer structural patterns.

Defense mechanisms. Proteus implements *active defense* by varying its encryption structure over time, as shown in Fig. 1. Per-epoch key and selector regeneration, combined with region-aware encryption, deny the attacker a stable cryptographic mapping between ciphertext and plaintext. Even under key exposure, an adversary must recover both the current epoch key and selector sequence to decrypt data. Continuous churn further disrupts any correlation attempts based on long-term ciphertext observation.

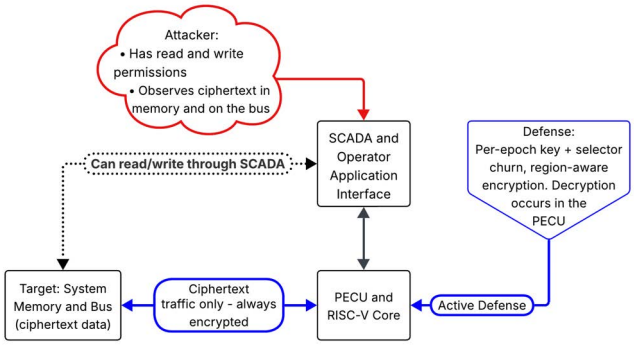


Fig. 1. Simplified threat model showing attacker access via SCADA, ciphertext-only observation, and *Proteus* defenses employing per-epoch key churn and region-aware encryption.

III. Methodology

The methodology of this work extends the Morpheus II architecture with the proposed Proteus encryption framework, which introduces dynamic cipher reconfiguration as a hardware-level MTD. *Proteus* employs a rotation-set mechanism that enhances key-exposure resilience through a selector parameter controlling the active rotation schedule. As shown in Figure 2, this mechanism is embedded in the cipher's round function, which extends SIMON's datapath with a selector-driven multiplexer network. Each round varies its rotation constants according to a per-epoch schedule while retaining SIMON's lightweight structure. This design reintroduces the proactive churn behavior of Morpheus I [16], periodically regenerating encryption keys and selector schedules to sustain continuous protection. Collectively, these mechanisms transform Morpheus II's reactive encryption model into a continuously adaptive, hardware-efficient MTD suitable for persistent IoT deployments.

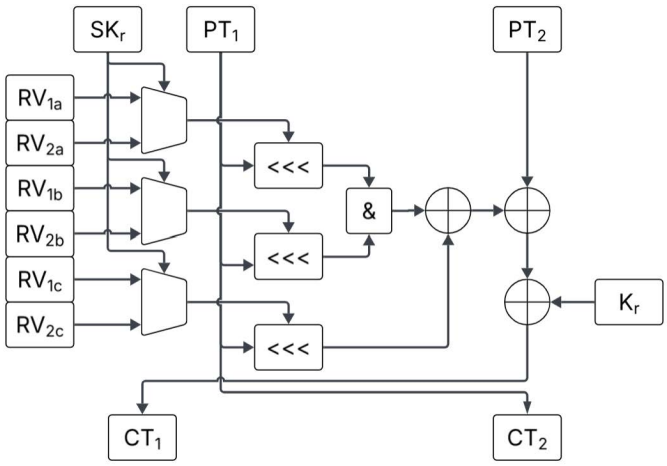


Fig. 2. Round structure of the *Proteus* cipher modifications. The round selector key (SK_r) determines the active rotation set for each round, selecting among the predefined rotation values (RV_{1a}/RV_{2a}, RV_{1b}/RV_{2b}, RV_{1c}/RV_{2c}). These rotation values define the bitwise shifts applied to the left plaintext word (PT_1). After rotation and logical operations (AND and XOR), the output is combined with the round key (K_r) to produce ciphertext words (CT_1, CT_2). This structure extends the SIMON round function by introducing dynamic, per-round rotation parameters determined by the selector key, enabling structural variability across epochs.

Building on this foundation, the overall system architecture of the proposed platform is illustrated in Figure 3. The diagram highlights how the *Proteus* extensions integrate into the Morpheus II baseline design. Original Morpheus II components, shown in black, include the RISC-V core, cache hierarchy, and memory interface. Newly introduced *Proteus* modules, shown in blue, provide per-epoch key and selector generation, churn control, and modified cache-encryption ciphers that collectively enable dynamic moving-target protection.

At the core of this architecture is the **Proteus Encryption and Churn Unit (PECU)**, which manages key generation, cache encryption, and epoch-level re-encryption, as shown in Figs. 3 and 4.

1) **Key and Selector Initialization:** At the start of each epoch, the PECU uses the on-chip TRNG to generate new encryption keys and selector schedules, which are loaded into the rotation-selector and key registers.
2) **Cache Encryption:** The I-Cache and D-Cache employ the *Proteus* cipher cores to encrypt all cache lines during normal operation, ensuring that only ciphertext leaves the trusted hardware domain (Figs. 3, 4).
3) **Epoch Transition Trigger:** When the churn timer signals a new epoch, the PECU initiates key regeneration and prepares the system for cache and memory re-encryption.
4) **In-Place Re-Encryption:** During transition, the PECU decrypts previous-epoch ciphertext within the protected domain and immediately re-encrypts it using the new keys, preventing any plaintext exposure.
5) **System-Wide Update:** The PECU coordinates with the memory control bus to propagate the refreshed cipher-

Authorized licensed use limited to: University of Ulsan. Downloaded on July 17,2026 at 02:19:57 UTC from IEEE Xplore. Restrictions apply.

text across all external memory regions, maintaining consistent encryption under the new epoch.

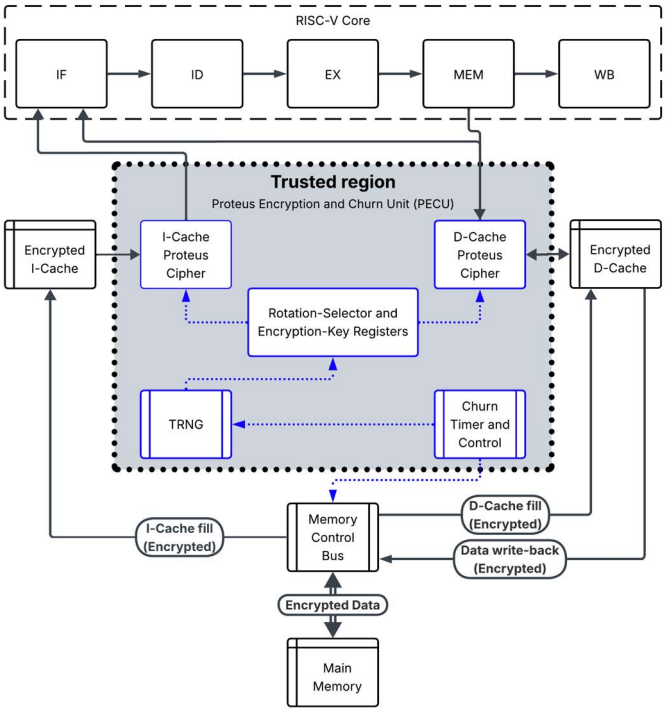


Fig. 3. System architecture showing how *Proteus* integrates into the Morpheus II baseline during normal execution. Black components represent the original Morpheus II design, and blue components appear within the dotted boundary denoting the trusted Proteus Encryption and Churn Unit (PECU). All decryption, key handling, and selector management occur strictly inside this region and are never accessible to software or external components. This figure illustrates steady-state operation under continuous ciphertext-only handling outside the trusted domain.

The runtime behavior of the platform during epoch transitions is illustrated in Figure 4. At each epoch boundary, the system derives fresh encryption keys and selector schedules from the on-chip true random number generator (TRNG), enabling continuous cryptographic renewal. During this process, the instruction cache (I-Cache) and data cache (D-Cache) function as secure buffers for ciphertext, ensuring that no plaintext is ever held or exposed. Ciphertext from the previous epoch is first loaded into the I-Cache, where it is decrypted by the prior-epoch cipher and immediately re-encrypted by the current-epoch cipher before being written to the D-Cache. The PECU performs this operation by briefly disconnecting the RISC–V core from the memory bus, routing control through the re–encryption datapath, and then restoring normal execution. The resulting pause cannot be compared to Morpheus II, as its implementation details are not publicly disclosed. The orange path in Figure 4 illustrates this re–encryption process, showing how *Proteus* preserves ciphertext integrity and confidentiality across epoch transitions without exposing intermediate plaintext.

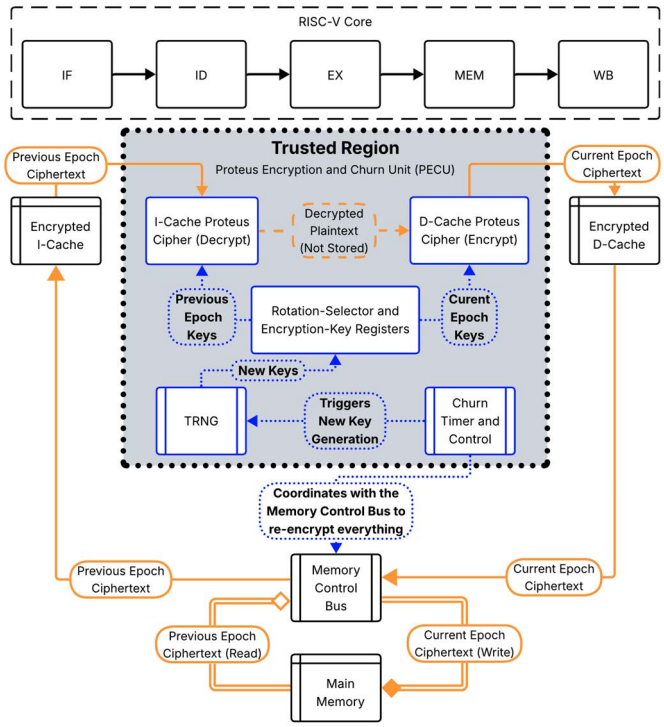


Fig. 4. *Proteus* epoch-transition and re-encryption process. Black elements show the Morpheus II baseline, while blue elements and dashed paths lie within the dotted trusted region (PECU). Orange arrows depict the controlled re-encryption flow during a churn event: prior-epoch ciphertext is decrypted only inside the trusted region and immediately re-encrypted under new keys. Secret state never leaves this boundary, ensuring that plaintext and key material remain inaccessible to software and external components.

A. Note on Region-Aware Encryption Mapping

Proteus can support region-aware encryption mapping, a potential extension designed to enhance resilience against ciphertext reuse and code-reuse attacks. In this scheme, each memory region is bound to a specific cipher configuration within an epoch so that every block in region r is encrypted and decrypted only under the rotation-set and selector schedule assigned to r. Formally, let e denote the current epoch and let $\mathcal{R}$ be a partition of the physical address space; a region-aware mapping is a function

$$f : \mathcal{R} \times \{e\} \to \mathcal{S}, \quad (r, e) \mapsto S_e(r),$$

where $\mathcal{S}$ is the set of valid selector schedules or rotation-set identifiers. In *Proteus*, $S_e(r)$ would select the active rotation set per round for all blocks in region r during epoch e, with f refreshed at each epoch boundary. This spatial binding would prevent ciphertext relocation attacks, since moving a block from r_i to r_j without re-encryption would produce invalid plaintext or detectable faults. It would also enlarge the adversary's search space by multiplying selector schedules across regions. At the same time, it can preserve SIMON compatibility and lightweight hardware cost by implementing f through a small per-region table or by deriving $S_e(r)$ from the region identifier and epoch seed. Although this capability

Authorized licensed use limited to: University of Ulsan. Downloaded on July 17,2026 at 02:19:57 UTC from IEEE Xplore. Restrictions apply.

was not evaluated in the present study, it represents a viable direction for future work to extend the MTD properties of *Proteus*.

B. Key and Selector Management, Search Space, and Churn

The *Proteus* framework treats encryption keys and selector parameters as independent secrets to strengthen resistance to key exposure. Each epoch generates a new selector stored separately from the key, producing an exponential number of possible schedules (for example, Proteus-2 with $R = 2$ and $N = 12$ yields $2^{12} = 4{,}096$ schedules and Proteus-4 with $R = 4$ yields $4^{12} = 16{,}777{,}216$), and mapping selectors to distinct memory regions multiplies this space further, making full-memory recovery infeasible even with key exposure; both secrets refresh every epoch, preventing decryption of prior ciphertexts, while *Proteus* preserves SIMON's key schedule and confines randomization to rotation-set selection for simplicity and compatibility.

C. Design Objectives

The design of *Proteus* was guided by three primary objectives: (1) preserve near-SIMON efficiency (i.e., incur only modest overhead) for lightweight hardware implementations, (2) incorporate MTD principles through randomized rotation sets, and (3) achieve hardware realizability with minimal additional overhead.

D. Rotation Set Selection

Rotation sets have been selected based on configurations identified in [14] that exhibit strong diffusion properties, defined as rotation-constant triplets (α, β, γ) that propagate a single active bit across the cipher state in the fewest rounds. In this work, we focus on diffusion in the context of differential propagation. An offline MATLAB script has been used to systematically evaluate candidate triplets and confirm rapid state expansion under this model. To encourage faster diffusion in early rounds, distinct rotation sets are employed in the first two rounds.

E. Round Function Construction

Proteus retains the Feistel structure of SIMON [2], whose round function is defined as

$$F(x) = \big((x \lll \alpha) \,\&\, (x \lll \beta)\big) \oplus (x \lll \gamma), \tag{1}$$

where $\lll$ denotes a circular left rotation by the specified number of bit positions, $\&$ is bitwise AND, and $\oplus$ is bitwise XOR. The constants (α, β, γ) define the rotation parameters.

Proteus generalizes this structure by introducing multiple candidate rotation sets that are dynamically selected at runtime. In the Proteus-2 configuration, two rotation sets are defined as

$$S_A = (R_1, R_4, R_5), \quad S_B = (R_1, R_5, R_4),$$

and a selector bit S, derived from the per-epoch schedule, determines the active set in each round. The combined round function is therefore expressed as

$$\begin{aligned} F(x) = (\neg S \,\&\, (((x \lll R_1) \,\&\, (x \lll R_4)) \oplus (x \lll R_5))) \\ \oplus (S \,\&\, (((x \lll R_1) \,\&\, (x \lll R_5)) \oplus (x \lll R_4))), \end{aligned} \tag{2}$$

where $\neg S$ denotes logical negation of the selector bit. This form represents a selector-controlled branching mechanism that dynamically alternates between the two rotation sets within each round.

For implementation clarity, the selector-controlled round function can be equivalently expressed using a 2:1 multiplexer:

$$\begin{aligned} F(x) = \big((x \lll R_1) \,\&\, \mathrm{MUX}\big(S,\, (x \lll R_4),\, (x \lll R_5)\big)\big) \\ \oplus \mathrm{MUX}\big(S,\, (x \lll R_5),\, (x \lll R_4)\big), \end{aligned} \tag{3}$$

where MUX denotes a 2:1 multiplexer controlled by the selector bit S. When $S = 0$, the multiplexer selects the rotation set (R_1, R_4, R_5); when $S = 1$, it selects (R_1, R_5, R_4). This formulation highlights the hardware realization of the selector mechanism, which allows *Proteus* to alternate between rotation sets through simple multiplexing logic rather than separate functional units. By introducing this dynamic selection at the round level, *Proteus* transforms SIMON's static round function into a configurable primitive that adapts continuously to the selector schedule, enabling fine-grained structural variability while preserving lightweight efficiency.

From a hardware standpoint, SIMON's fixed rotations are implemented in parallel, which during FPGA or ASIC synthesis operation can lead to logic duplication by the electronic design automation (EDA) tool when fanout exceeds local routing capacity. In *Proteus*, shared rotation constants are computed once and reused across both branches, allowing synthesis tools to merge redundant logic into a single datapath with lightweight multiplexers on the rotation inputs. Consequently, the overhead of supporting multiple rotation sets depends primarily on the degree of constant overlap.

In summary, *Proteus* extends SIMON by introducing round-level structural variability through dynamic rotation-set selection. This approach preserves SIMON's lightweight efficiency while providing greater algorithmic diversity, as reflected in the synthesis results in Table II.

F. Round Function Example

To illustrate the effect of rotation-set variability, consider the Proteus-2 configuration, which defines two rotation sets: $(1, 4, 5)$ and $(1, 5, 4)$. Encryption begins from a common plaintext with round keys `0x1918` and `0x3230` applied in the first and second rounds, respectively. At each round, a selector bit determines which rotation set is active. As shown in Figure 5, this produces a binary branching structure in which each round doubles the number of possible ciphertext trajectories. After two rounds, the plaintext evolves into four distinct intermediate states, demonstrating how Proteus-2 introduces exponential variability under a fixed key.

Authorized licensed use limited to: University of Ulsan. Downloaded on July 17,2026 at 02:19:57 UTC from IEEE Xplore. Restrictions apply.

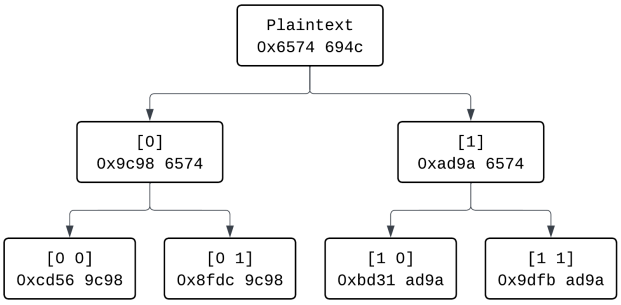


Fig. 5. Binary tree expansion of a 2-set *Proteus* cipher after two rounds with keys `0x1918` and `0x3230`. Two rotation sets, $(1, 4, 5)$ and $(1, 5, 4)$, yield 2^r possible ciphertexts after r rounds.

As the number of available rotation sets increases, the number of possible ciphertexts grows exponentially with the number of rounds. Specifically, for s rotation sets and r rounds, the total number of distinct ciphertexts is s^r. For example, using 2, 4, 8, and 16 rotation sets produces $2^6 = 64$, $4^6 = 4{,}096$, $8^6 = 262{,}144$, and $16^6 = 16{,}777{,}216$ ciphertext variations after six rounds, respectively. This rapid combinatorial expansion amplifies the MTD property by generating a substantially larger set of valid ciphertexts under the same key, thereby increasing the cipher's structural unpredictability.

IV. Security Model Derived From Churn Cost and Rotation Diversity

This section introduces a general mathematical model that connects the churn cost, the number of rounds, the number of rotation schedules, the number of rotation seeds, and the epoch duration to the protection that Proteus provides. The goal of this model is to quantify how much memory can be re-encrypted in a given time, how many instructions remain protected within an epoch, how rotation schedule diversity limits the effect of configuration exposure, and how churn reduces the probability that an attacker completes an exploit.

A. Churn Time and Protection Window

Let S_{main} be the size of the protected main memory region in bytes. The churn mechanism processes memory in units of size S_I bytes from the I-cache interface. The number of blocks that must be processed during a complete churn is

$$N = \left\lceil \frac{S_{\text{main}}}{S_I} \right\rceil \tag{4}$$

The memory system provides sustained read and write bandwidths B_{rd} and B_{wr}, with latencies L_{rd} and L_{wr} that include the fixed cost of transferring a full cache block. The time to read an instruction block of size S_I and to write a data block of size S_D is

$$t_{\text{fetch}} = L_{\text{rd}} + \frac{S_I}{B_{\text{rd}}} \qquad \text{and} \qquad t_{\text{write}} = L_{\text{wr}} + \frac{S_D}{B_{\text{wr}}} \tag{5}$$

A re-encryption performs R_{dec} decrypt rounds and R_{enc} encrypt rounds; if each round requires c_{round} cycles and the core frequency is f_c, then the total number of rounds and the associated round-processing time are

$$R_{\text{tot}} = R_{\text{dec}} + R_{\text{enc}}, \qquad t_{\text{rounds}} = \frac{R_{\text{tot}}\, c_{\text{round}}}{f_c} \tag{6}$$

Streaming is limited by the smaller cache-line size $\min(S_I, S_D)$; with datapath width W_c bytes per cycle and core frequency f_c, the streaming time and total cipher-processing time per block are

$$t_{\text{stream}} = \frac{\min(S_I, S_D)}{W_c f_c}, \qquad t_{\text{cipher}} = t_{\text{rounds}} + t_{\text{stream}} \tag{7}$$

A fixed controller overhead per block is modeled by t_{ctrl}. In full-duplex mode, the system overlaps fetching, cipher processing, and writing a block with the resulting per-block churn time denoted by $t_{\text{unit}}^{(\text{full})}$. In contrast, time-multiplexed mode serializes these stages, and the corresponding per-block churn time is $t_{\text{unit}}^{(\text{tm})}$. These timings are

$$t_{\text{unit}}^{(\text{full})}(R_{\text{tot}}) = \max\bigl(t_{\text{fetch}}, t_{\text{cipher}}(R_{\text{tot}}), t_{\text{write}}\bigr) + t_{\text{ctrl}} \tag{8}$$

$$t_{\text{unit}}^{(\text{tm})}(R_{\text{tot}}) = t_{\text{fetch}} + t_{\text{cipher}}(R_{\text{tot}}) + t_{\text{write}} + t_{\text{ctrl}} \tag{9}$$

A complete churn processes N cache blocks. The total churn time T_{churn} depends on the per-block processing time t_{unit}, which is selected from either $t_{\text{unit}}^{(\text{full})}$ or $t_{\text{unit}}^{(\text{tm})}$ according to the operating mode. The overall churn time is

$$T_{\text{churn}} = N\, t_{\text{unit}} \tag{10}$$

Let τ denote the epoch duration. The quantity $\rho(\tau)$ represents the fraction of protected memory that is re-encrypted at least once during an epoch; it captures how effectively the system achieves moving-target protection over time. This fraction is determined by the ratio between the epoch duration and the full-churn time, yielding

$$\rho(\tau) = \min\left(1, \frac{\tau}{T_{\text{churn}}}\right) \tag{11}$$

If each instruction occupies w_{inst} bytes, then the number of instructions effectively refreshed per epoch is determined by the fraction of memory re-encrypted during that epoch, given by $\rho(\tau)$, and the total protected memory size S_{main}. The number of refreshed instructions is

$$N_{\text{inst}}(\tau) = \frac{\rho(\tau)\, S_{\text{main}}}{w_{\text{inst}}} \tag{12}$$

Authorized licensed use limited to: University of Ulsan. Downloaded on July 17,2026 at 02:19:57 UTC from IEEE Xplore. Restrictions apply.

B. Rotation Schedule Diversity and Blast Radius

Rotation schedules create spatial diversity by varying how different regions of protected memory are encrypted. Let R_{sets} be the number of rotation sets and N_{round} the number of rounds per encryption. These determine the total number of possible schedules, and the subset used in an epoch is C_{used}. The maximum schedule count and its allowable range are

$$C_{\max} = R_{\text{sets}}^{N_{\text{round}}}, \qquad 1 \le C_{\text{used}} \le C_{\max} \tag{13}$$

Memory is modeled as M logical blocks of size B_{blk} bytes. The total number of blocks is M, and these blocks are divided into L rotation-seed domains, with domain ℓ containing M_ℓ blocks (balanced when $M_\ell \approx M/L$). The block count and domain partitioning are

$$M = \left\lceil \frac{S_{\text{main}}}{B_{\text{blk}}} \right\rceil, \qquad \sum_{\ell=1}^{L} M_\ell = M \tag{14}$$

Within each rotation-seed domain, blocks are assigned to rotation schedules using a uniform random mapping. Let M_ℓ be the number of blocks in domain ℓ, and let C_{used} be the number of schedules used in an epoch. We write $\mathbb{E}[\cdot]$ for expected value. Under this model, the expected number of blocks in domain ℓ that use a compromised schedule, and the fraction of exposed blocks in that domain, are

$$\mathbb{E}[K_\ell] = \frac{M_\ell}{C_{\text{used}}}, \qquad \beta_{\ell,\text{sched}} = \frac{1}{C_{\text{used}}} \tag{15}$$

Across the full memory region, compromising one schedule exposes the same fraction of blocks. This fraction ranges from complete exposure when $C_{\text{used}} = 1$ to roughly one block when $C_{\text{used}} \ge M$, where M is the total number of blocks. The expected exposed fraction is

$$\beta_{\text{sched}} = \frac{1}{C_{\text{used}}} \tag{16}$$

If an attacker compromises a rotation seed, all blocks in that seed domain are exposed. Let $M_{\ell^\star}$ denote the number of blocks in the compromised domain and let M be the total number of blocks. The resulting blast-radius fraction is

$$\beta_{\text{seed}} = \frac{M_{\ell^\star}}{M}, \qquad \beta_{\text{seed}} \approx \frac{1}{L} \text{ (balanced)} \tag{17}$$

If all rotation seeds and schedules are compromised, the entire region is exposed, giving $\beta_{\text{full}} = 1$.

C. Combined Temporal and Spatial Exposure Model

We relate churn frequency and rotation diversity to attacker success. Let X be the time an attacker needs to complete an exploit under a fixed configuration, and let $F(t)$ be its cumulative distribution function:

$$F(t) = \Pr[X \le t] \tag{18}$$

In a static system, the probability of success within time horizon T is

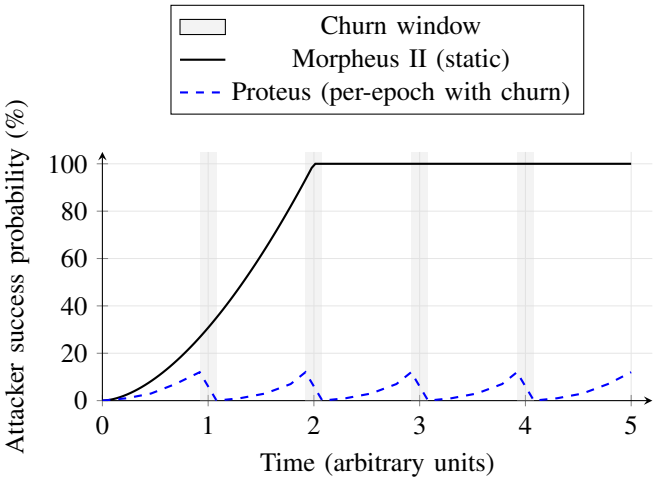


Fig. 6. This figure provides a conceptual, non-definitive illustration of attacker success probability over time. Morpheus II employs a static configuration, leading to steadily increasing attacker success. Proteus incorporates periodic churn and shows stronger per-epoch resistance, producing slower growth within each epoch and a reset to near zero during each churn window. *Note:* Morpheus II resets its configuration only on detected faults; this plot assumes no faults occur. All axis values are illustrative.

$$P_{\text{static}}(T) = F(T) \tag{19}$$

With periodic churn, the attacker must succeed within a single epoch of length τ, giving per-epoch success probability $F(\tau)$. Over a horizon T containing $n(T) = \lfloor T/\tau \rfloor$ epochs, the probability of at least one successful attempt is

$$P_{\text{MTD}}(T) = 1 - \big(1 - F(\tau)\big)^{n(T)} \tag{20}$$

The temporal gain from churn measures how much periodic resets reduce attacker success when some exploits require more than τ seconds. It is defined as

$$G_{\text{time}}(T, \tau) = \frac{P_{\text{static}}(T)}{P_{\text{MTD}}(T)} \tag{21}$$

To account for spatial effects, let β be the fraction of memory exposed after a configuration compromise, and let $\rho(\tau)$ be the fraction re-encrypted per epoch. The effective blast radius is

Overall, the churn-time, spatial, and temporal models show how Proteus limits an adversary's ability to accumulate progress across epochs. Churn bounds how much state can be refreshed, rotation diversity constrains the impact of configuration exposure, and periodic resets reduce long-term exploit success. Figure 6 illustrates these combined effects, contrasting the monotonic success growth of a static configuration with the reset-and-regrow behavior introduced by Proteus.

V. Evaluation

A. Statistical Randomness Testing

To evaluate the statistical quality of ciphertext, the NIST Statistical Test Suite (STS) [19] was executed on data

Authorized licensed use limited to: University of Ulsan. Downloaded on July 17,2026 at 02:19:57 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NIST STS RESULTS FOR PROTEUS-2 (12 KEYS, 30 RANDOMIZED A/B SCHEDULES PER KEY)

Test	Pass Rate	Outcome
All STS (overall)	356–359 / 360 ($\geq$350)	Pass
Weakest (Non-Overlapping Template)	352 / 360 ($\geq$350)	Pass
Random Excursions / Variant	228–234 / 234 ($\geq$227)	Pass

Note: Each of the 12 keys was tested under 30 randomized A/B rotation-set schedules, where $A = (1, 4, 5)$ and $B = (1, 5, 4)$, producing 360 total sequences. All tests exceeded NIST minimum pass-rate thresholds.

generated by both SIMON and *Proteus*. The baseline SIMON (1,8,2) with 12 rounds was tested across 12 sequences of 1,000,000 bits each, using 12 distinct keys. All tests met the NIST minimum pass rate of 10/12 sequences, although some cases (particularly the frequency and non-overlapping template tests) only marginally satisfied this threshold, which is consistent with SIMON's limited statistical margin at reduced round counts [4], [10], [20].

Under identical conditions, ciphertext from the *Proteus*-2 configuration was tested across 360 randomized A/B rotation-set sequences, and the results in Table I show all categories comfortably exceeding NIST thresholds, indicating that rotation-set variability preserves, and in practice enhances, diffusion and randomness relative to reduced-round SIMON.

B. Hardware Results

Proteus was implemented in `SystemVerilog` and synthesized on an Intel MAX10 FPGA (10M50DAF484C7G) using Quartus Prime Lite 24.1std with balanced synthesis. Twelve unrolled rounds were used to achieve single-cycle throughput, and critical-path delay was measured from F_{max} reports.

Table II shows that 12-round, 32-bit implementations of SIMON and *Proteus* achieve efficient hardware operation, with selector-controlled rotation switching adding only modest area and minimal frequency loss relative to the Morpheus II SIMON (1,8,2) baseline; Proteus-2 demonstrates fine-grained rotation-set reconfiguration within a lightweight core, and Proteus-4 extends this to multiple sets, confirming scalability for epoch-driven cipher adaptation.

To evaluate the security–hardware trade-off, we systematically swept rotation constants while fixing all others. The parameter (R_α) was held constant because it appears symmetrically with (R_β) in the same AND term, so changing it has little hardware impact. With (R_α) fixed, (R_β) was varied from 0–15 while (R_γ) was held constant, and then (R_γ) was swept with (R_α) and (R_β) fixed. Each configuration was synthesized and compared against the strongest sets from prior work [14]. The results show that shared or aligned rotations reduce logic and routing cost while preserving strong diffusion, whereas distinct constants increase multiplexing complexity and area, confirming that rotation selection can balance strength and efficiency.

Table II also shows that Proteus-4 attains slightly higher frequency and throughput than Proteus-2 despite using more LUTs. This is due to its rotation constants: several include a rotation by eight, which aligns naturally with the Intel MAX10 FPGA's logic array block (LAB) structure, enabling more efficient routing and shorter critical-path delay. The synthesis tool can therefore offset the added multiplexing overhead. Proteus-2 lacks this alignment and runs at a lower frequency. Overall, performance and area depend not only on the number of rotation sets but also on the specific rotation values and how they interact with the underlying hardware fabric.

All synthesis results correspond to a single *Proteus* encryption core and quantify implementation cost under identical synthesis conditions. Core-level evaluation enables a direct comparison of area, frequency (F_{max}), and throughput relative to SIMON, independent of higher-level re-encryption logic.

Overall, the data in Table II confirm that algorithmic moving-target defense can be achieved within the lightweight design envelope of SIMON. Although *Proteus* introduces a small area increase and minor frequency reduction, it provides measurable gains in cryptanalytic resistance. This trade-off demonstrates that continuous, cipher-level reconfiguration is practical for embedded platforms that prioritize resilience over minimal resource usage. Proteus-4 was evaluated only for hardware efficiency and was not included in the statistical randomness testing discussed earlier.

C. Preliminary Cryptanalysis

This subsection evaluates *Proteus* using differential cryptanalysis, which examines how a chosen difference between two plaintexts, $\Delta P = P \oplus P'$, influences the resulting ciphertext difference, $\Delta C = C \oplus C'$, under the same key. In an ideal cipher, differences spread in a way that appears random, so no particular output difference should occur more frequently than any other. If structural patterns exist, however, certain input differences can produce output differences with higher-than-random probability, revealing exploitable bias or distinguishing the cipher from a random permutation.

TABLE II
HARDWARE RESULTS AND ROTATION-SET CONFIGURATIONS FOR *Proteus* VARIANTS (INTEL MAX10 FPGA, 32-BIT BLOCK, 12 UNROLLED ROUNDS)

Cipher (Set)	LUTs	FFs	Freq. (MHz)	Rate (Mbps)	Area/Rate (LUTs/Mbps)	Rotation Sets $(R_\alpha, R_\beta, R_\gamma)$
SIMON (1,8,2)	658	456	46.82	1,498	0.439	Baseline $(1, 8, 2)$
Proteus-2 (A)	871	480	40.54	1,297	0.672	$(1, 4, 5)$, $(1, 5, 4)$
Proteus-4 (B)	1,295	504	43.79	1,401	0.924	$(1, 4, 5)$, $(1, 5, 4)$, $(1, 4, 8)$, $(1, 5, 8)$

Note: Proteus-4 achieves slightly higher frequency than Proteus-2 due to rotation-by-8 values aligning with MAX10 FPGA logic array block (LAB) boundaries, which improves routing efficiency and shortens the critical path.

Authorized licensed use limited to: University of Ulsan. Downloaded on July 17,2026 at 02:19:57 UTC from IEEE Xplore. Restrictions apply.

We summarize resistance using the `total_log2p` values produced by the analysis tool (tool from reference [8]) and define *strength* as $-$`total_log2p` so that larger values correspond to lower differential probabilities and therefore stronger resistance.

Because enumerating all 2^{32} possible input differences is impractical, we use a modified version of the differential analysis tool introduced by Beierle et al. [8], extended to support the selector-driven rotation schedules of *Proteus*. Rather than analyzing the full difference space, we restrict attention to a subset of bit positions within each 16-bit word using a window mask containing w active bits. This reduces the effective state space to 2^w representable differences per half while still allowing explicit probability propagation across rounds.

The tool in [8] constructs all differences supported by the window mask and propagates a probability distribution over pairs of left and right half-word differences through the rounds according to the selected rotation schedule. For each round, it computes how input differences evolve through the nonlinear function and assigns probabilities that negative powers of 2. After the final round, it reports the most likely single-bit output difference within the window along with its $\log_2$ probability, which serves as the primary comparison metric.

During early exploration of the full $2^{12} = 4096$ schedule space, we observed that the measured differential strength values repeat every 16 configuration keys. This suggests structural periodicity in how the selector bits interact with the specific rotation sets $(1, 4, 5)$ and $(1, 5, 4)$ used in this study. Because of this repetition, evaluating 16 consecutive configuration keys captures one complete cycle of the observed behavior and provides a representative view without exhaustively sweeping all 4096 possibilities. The results also exhibit a repeating pattern with a period of four keys, reflecting symmetry within the schedule space for this particular rotation-set pair. We note that this periodic structure may be a property of the chosen rotation sets rather than an inherent property of all possible rotation-set combinations.

Figure 7 shows the minimum, mean, and maximum differential strength across the tested rotation-set keys compared to the baseline. While *Proteus* improves differential resistance on average, the periodic dips in the figure reflect the underlying symmetry in the rotation schedule space and highlight the importance of selecting rotation sets that maximize structural diversity rather than relying solely on individual diffusion strength. As stated earlier the rotation key selection that are weaker than the baseline Simon can be excluded, resulting in better security for our proposed Proteus design.

VI. Discussion

Proteus demonstrates that incorporating MTD principles into lightweight block cipher design is both feasible and effective. This section discusses the cryptanalytic findings, their security implications, comparisons with prior MTD architectures, and key observations on algorithm–hardware co-design.

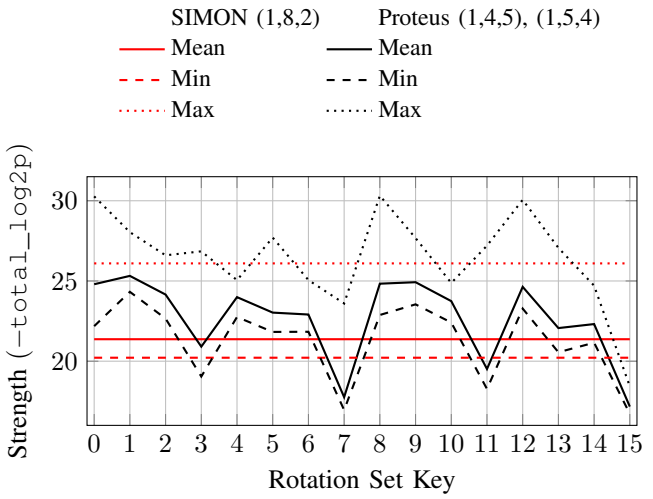


Fig. 7. Differential strength across 16 rotation-set configuration keys for Proteus using rotation sets (1,4,5) and (1,5,4), compared to the SIMON (1,8,2) baseline. Strength is defined as $-$`total_log2p` so that larger values correspond to lower differential probability. Minimum, mean, and maximum values are shown, illustrating the overall improvement over SIMON as well as a repeating pattern in the configuration space, including a weaker trough that recurs approximately every four keys.

A. Security Implications

These results highlight the potential for lightweight ciphers to address key-exposure threats often overlooked in designs assuming perfect key secrecy. When multiple selector schedules are employed within the same message, different plaintext segments are encrypted under distinct rotation paths, so key disclosure reveals only a subset of ciphertext relationships. This property mitigates single-point failures and supports further exploration of randomized round functions in resource-constrained environments.

B. Comparison with Prior MTD Systems

A key distinction is that *Proteus* introduces proactive churn at the cipher level, unlike Morpheus I's microarchitectural churn and Morpheus II's static 12-round SIMON core, by alternating among multiple rotation sets under a per-epoch selector schedule, expanding the search space from $2^{12} = 4{,}096$ (Proteus-2) to $4^{12} = 16{,}777{,}216$ (Proteus-4) and providing stronger moving-target protection with modest overhead.

C. Algorithm–Hardware Integration and Security Impact

Proteus illustrates how algorithmic flexibility can be natively supported in hardware to enhance resilience. Beyond the measured performance metrics, its primary contribution is structural: the cipher accommodates multiple rotation-set schedules, enabling the round function to vary dynamically over time. As outlined in the threat model, this adds a second protective dimension by requiring an adversary to recover both the symmetric key and the selector schedule defining active rotation sets. Consequently, key exposure alone cannot compromise system security and complicates cryptanalytic methods that depend on a fixed round structure.

Authorized licensed use limited to: University of Ulsan. Downloaded on July 17,2026 at 02:19:57 UTC from IEEE Xplore. Restrictions apply.

VII. Conclusion

This work presented *Proteus*, a Morpheus II extension that integrates algorithmic MTD into the encryption and churn pipeline. *Proteus* introduces per-epoch key and selector generation, selector-driven rotation sets within a SIMON-compatible core, and optional region-aware mapping, delivering proactive churn and fine-grained structural variability while preserving the lightweight design principles and interfaces of Morpheus II. Differential evaluation under a windowed analysis model demonstrated measurable improvements in resistance relative to the 12-round SIMON baseline, while also highlighting the importance of rotation-set diversity and schedule symmetry. An Intel MAX10 FPGA prototype demonstrates that *Proteus* achieves these capabilities with modest area growth and minimal throughput impact relative to SIMON, establishing a practical design space where algorithmic and architectural dynamism reinforce each other to harden systems against ciphertext reuse and key exposure. Overall, *Proteus* shows that continuous, cipher-level reconfiguration can be efficiently realized in lightweight platforms, offering a viable foundation for resilient MTD in long-lived IoT deployments.

References

[1] J. Daemen and V. Rijmen, *The Design of Rijndael: AES — The Advanced Encryption Standard*. Springer, 2002. [Online]. Available: https://link.springer.com/book/10.1007/978-3-662-04722-4

[2] R. Beaulieu *et al.*, "The SIMON and SPECK families of lightweight block ciphers," in *Proceedings of the 52nd Annual Design Automation Conference (DAC)*. New York, NY, USA: ACM, 2015. [Online]. Available: https://dl.acm.org/doi/10.1145/2744769.2747946

[3] W. Diehl *et al.*, "Comparison of hardware and software implementations of selected lightweight block ciphers," in *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2017. [Online]. Available: https://ieeexplore.ieee.org/document/8056808

[4] F. Abed, E. List, S. Lucks, and J. Wenzel, "Differential and linear cryptanalysis of reduced round SIMON," Cryptology ePrint Archive, Paper 2013/526, 2013. [Online]. Available: https://eprint.iacr.org/2013/526

[5] K. F. Khoo *et al.*, "Cryptanalysis of SIMON and SPECK families of block ciphers," Cryptology ePrint Archive, Paper 2013/543, 2013. [Online]. Available: https://eprint.iacr.org/2013/543

[6] Q. Wang *et al.*, "Cryptanalysis of reduced-round SIMON-32 and SIMON-48," Cryptology ePrint Archive, Paper 2014/761, 2014. [Online]. Available: https://eprint.iacr.org/2014/761

[7] M. A. Abdelraheem *et al.*, "Improved linear cryptanalysis of reduced-round SIMON-32 and SIMON-48," Cryptology ePrint Archive, Paper 2015/988, 2015. [Online]. Available: https://eprint.iacr.org/2015/988

[8] C. Beierle *et al.*, "Improved differential and linear cryptanalysis on round-reduced SIMON," Cryptology ePrint Archive, Paper 2025/178, 2025. [Online]. Available: https://eprint.iacr.org/2025/178

[9] C. Beierle, "Pen and paper arguments for SIMON and SIMON-like designs," Cryptology ePrint Archive, Paper 2016/651, 2016. [Online]. Available: https://eprint.iacr.org/2016/651

[10] I. Dinur, "Improved differential cryptanalysis of round-reduced SIMON and the choice of rotation constants," Cryptology ePrint Archive, Paper 2016/625, 2016. [Online]. Available: https://eprint.iacr.org/2016/625

[11] S. M. Dehnavi, "Further observations on SIMON and SPECK block cipher families," Cryptology ePrint Archive, Paper 2018/1107, 2018. [Online]. Available: https://eprint.iacr.org/2018/1107

[12] J. Lu *et al.*, "Improved rotational-xor cryptanalysis of SIMON-like block ciphers," Cryptology ePrint Archive, Paper 2022/402, 2022. [Online]. Available: https://eprint.iacr.org/2022/402

[13] K. Zhang, X. Lai, J. Guan, and B. Hu, "Selecting rotation constants on SIMON-type ciphers," *Journal of Database Management*, vol. 34, no. 1, pp. 1–23, 2023. [Online]. Available: https://dl.acm.org/doi/10.4018/JDM.318452

[14] K. Kondo, Y. Sasaki, and T. Iwata, "On the design rationale of SIMON block cipher: Integral attacks and impossible differential attacks against SIMON variants," in *Applied Cryptography and Network Security (ACNS) 2016*, ser. Lecture Notes in Computer Science, vol. 9696. Springer, 2016, pp. 518–536. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-39555-5_28

[15] J.-H. Cho *et al.*, "Toward proactive, adaptive defense: A survey on moving target defense," arXiv preprint arXiv:1909.08092, 2019. [Online]. Available: https://arxiv.org/abs/1909.08092

[16] M. Gallagher *et al.*, "Morpheus: A vulnerability-tolerant secure architecture based on ensembles of moving target defenses with churn," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. New York, NY, USA: ACM, 2019, pp. 469–484. [Online]. Available: https://dl.acm.org/doi/10.1145/3297858.3304037

[17] A. Harris *et al.*, "Morpheus II: A risc-v security extension for protecting vulnerable software and hardware," in *2021 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2021, pp. 1–12. [Online]. Available: https://ieeexplore.ieee.org/document/9702275

[18] D. Pliatsios, P. Sarigiannidis, T. Lagkas, and A. G. Sarigiannidis, "A survey on scada systems: Secure protocols, incidents, threats and tactics," *IEEE Communications Surveys Tutorials*, vol. 22, no. 3, pp. 1942–1976, 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9066892

[19] A. Rukhin *et al.*, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," National Institute of Standards and Technology (NIST), NIST Special Publication 800-22 Revision 1a, 2010. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-22r1a.pdf

[20] C. Niu, M. Li, J. Zhang, and M. Wang, "Improved differential and linear cryptanalysis on round-reduced SIMON," Cryptology ePrint Archive, Report 2025/178, 2025. [Online]. Available: https://eprint.iacr.org/2025/178

Authorized licensed use limited to: University of Ulsan. Downloaded on July 17,2026 at 02:19:57 UTC from IEEE Xplore. Restrictions apply.